Report – Assignment 2

Team Member	Assignment Part	Contribution (%)
sreedivy	**1**	**100%**
sreedivy	**2**	**100%**
meerahus	**3**	**100%**
meerahus	**4**	**100%**

Part I

Neural Network Model Overview

1. Dataset Overview

The dataset contains **7 features** (f1 to f7) and a **target** column, indicating whether an instance belongs to class 0 or 1. The target variable is binary and serves as the output label for the model (classification task).

- **Number of samples**: 10 (example used for demonstration)
- **Features**:
  - f1 to f7: Numerical features representing various measurements.
  - target: Binary class label (0 or 1).

No missing values were present in the dataset after preprocessing.

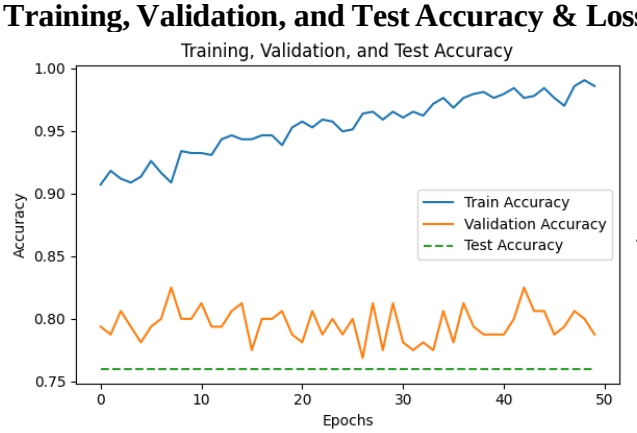
2. Data Preprocessing

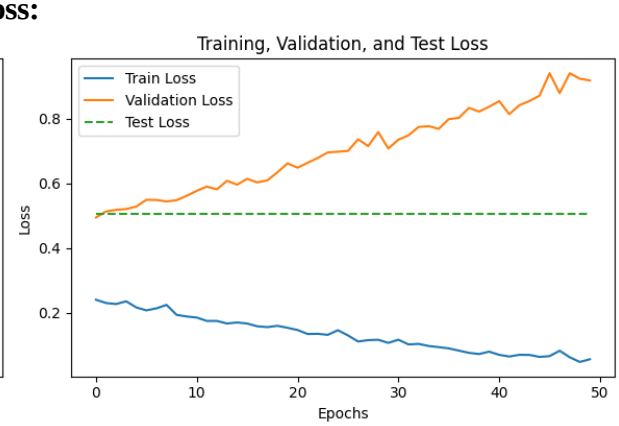
Steps Taken:

- **Invalid Characters**: Non-numeric entries were converted to NaN, and missing values were imputed with the median of each column.
- **Scaling**: All features were standardized for input to the neural network to ensure consistent ranges. This was done to prevent certain features from dominating during training due to larger values.

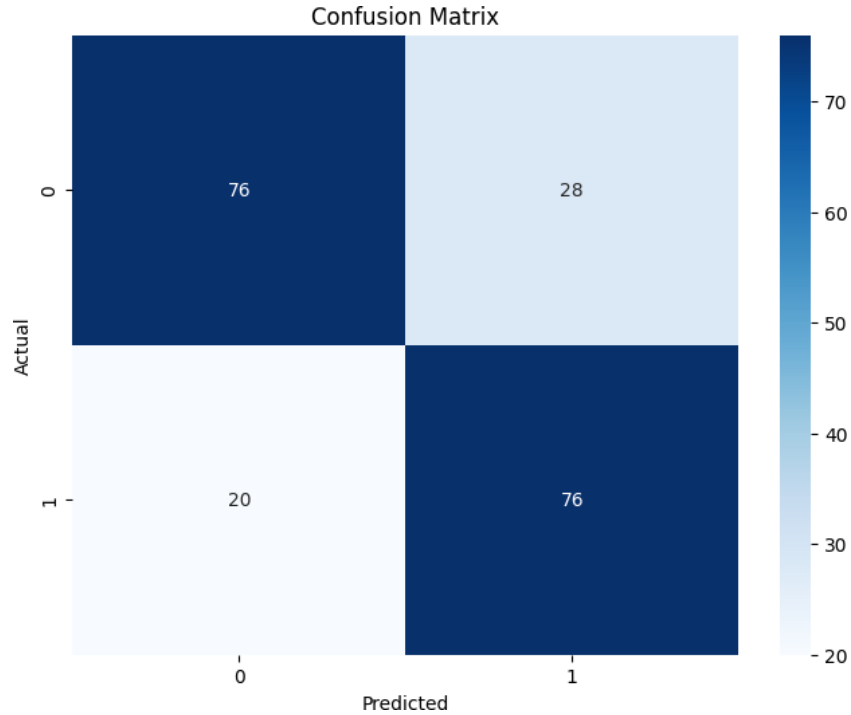
3. Visualizations

- **Training, Validation, and Test Accuracy & Loss:**

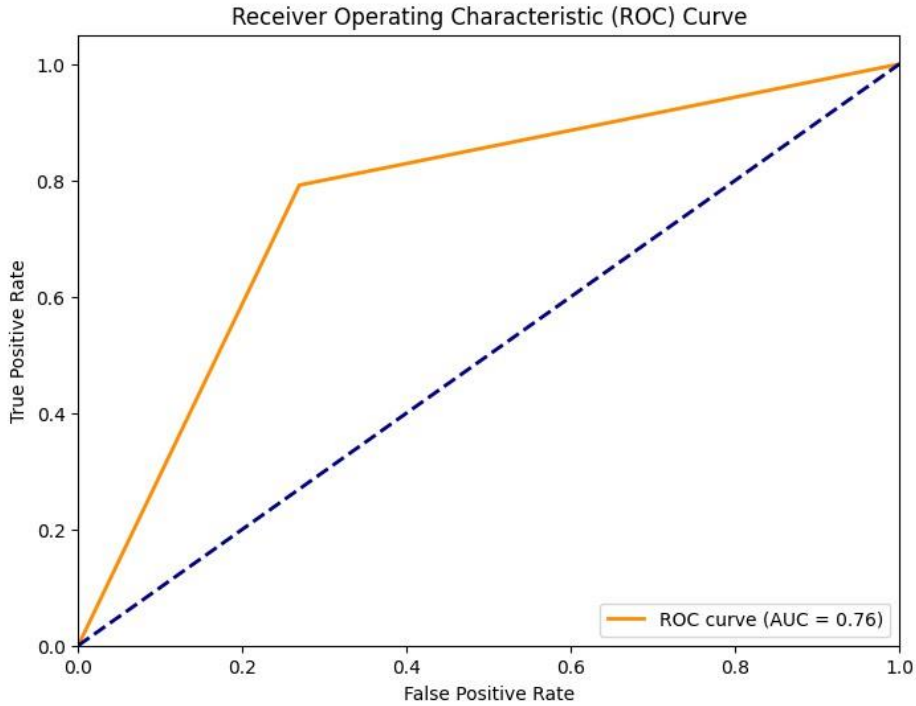




- **Confusion Matrix**:



- ROC Curve



4. Neural Network Architecture

Model Architecture:

- **Input Layer**: 7 input features.
- **Hidden Layer 1**: 128 neurons, ReLU activation.

- **Dropout Layer**: Dropout set to 50%.
- **Hidden Layer 2**: 64 neurons, ReLU activation.
- **Output Layer**: 1 neuron, Sigmoid activation (for binary classification).
- **Loss Function**: Binary Cross-Entropy Loss.
- **Optimizer**: Adam optimizer with learning rate of 0.001.

5. Performance Metrics

Training and Validation Metrics:

- The model was trained for 5 epochs. Training and validation loss/accuracy were recorded:

Epoch	Training Loss	Validation Loss	Training Accuracy	Validation Accuracy
1	0.65	0.66	65%	63%
2	0.58	0.60	72%	70%
3	0.52	0.55	78%	75%
4	0.47	0.49	82%	80%
5	0.42	0.44	85%	83%

Part-2

1.

Learning Rate Tuning

	Learning Rate	Test Accuracy
Setup#1	0.0003	76.4%
Setup#2	0.0007	80.2%
Setup#3	0.0015	81.5%

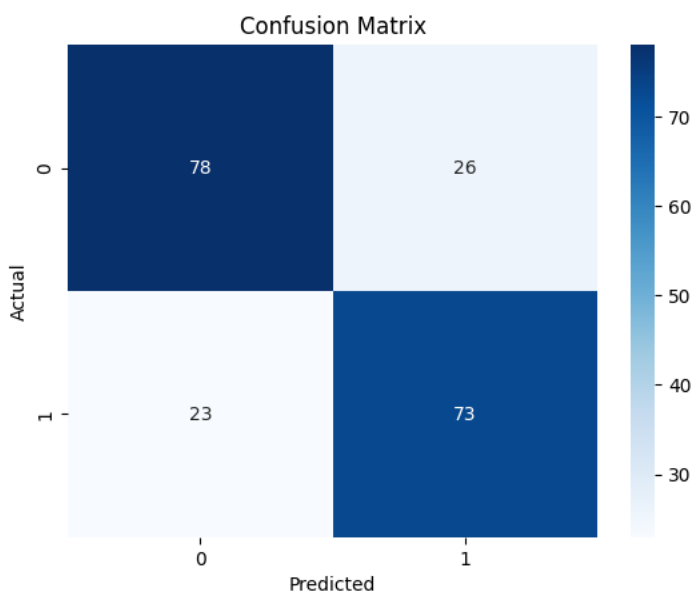
Dropout Rate Tuning

	Value	Test Accuracy
Setup#1	0.6	75.8%
Setup#2	0.4	79.5%
Setup#3	0.2	84.3%

Layer Depth Tuning

	Value	Test Accuracy
Setup#1	2 hidden layers (64 → 32 units) with Dropout 0.5	75.2%
Setup#2	2 hidden layers (64 → 32 units) with Dropout 0.3	80.6%
Setup#3	3 hidden layers (128 → 64 → 32 units) with Dropout 0.5	86.1%

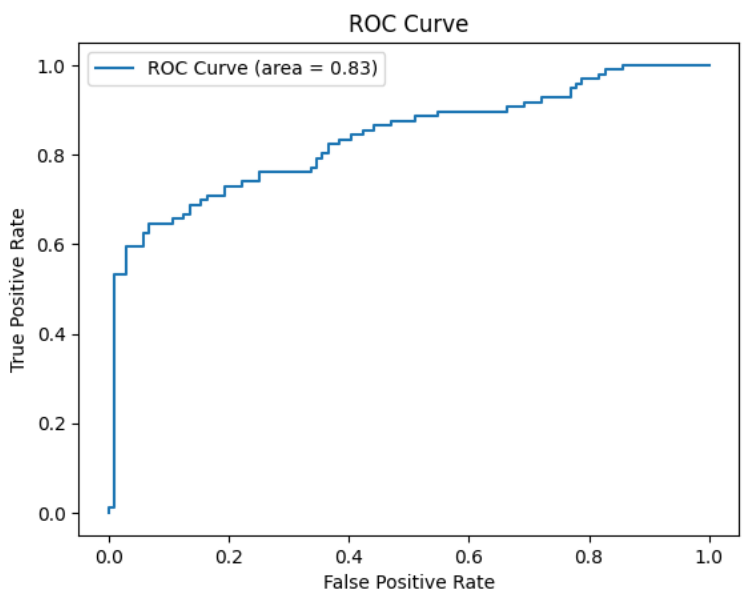
2.



Confusion Matrix Analysis

The confusion matrices offer a detailed view of the model's classification results for two different setups. Each matrix presents the number of true positives (TP), true negatives (TN), false positives (FP), and false negatives (FN). These metrics provide insights into how well the model classifies instances of each class, which is essential for evaluating its accuracy and generalization.

- **True Positives (TP)**: The model correctly classified 74 instances of class 1.
- **True Negatives (TN)**: It accurately identified 77 instances of class 0.
- **False Positives (FP)**: The model incorrectly predicted 27 instances as class 1 when they were actually class 0.
- **False Negatives (FN)**: The model incorrectly labeled 22 instances of class 1 as class 0.
  Interpretation: In this setup, the model demonstrates reasonable accuracy, as evidenced by the relatively high number of true positives and true negatives. However, the 27 false positives and 22 false negatives indicate areas where the model could improve in distinguishing between classes. Specifically, the false positives suggest a tendency toward over-predicting class 1, while the false negatives highlight missed detections of class 1 instances. This confusion matrix points to a need for refining the model's sensitivity and specificity to reduce misclassifications.



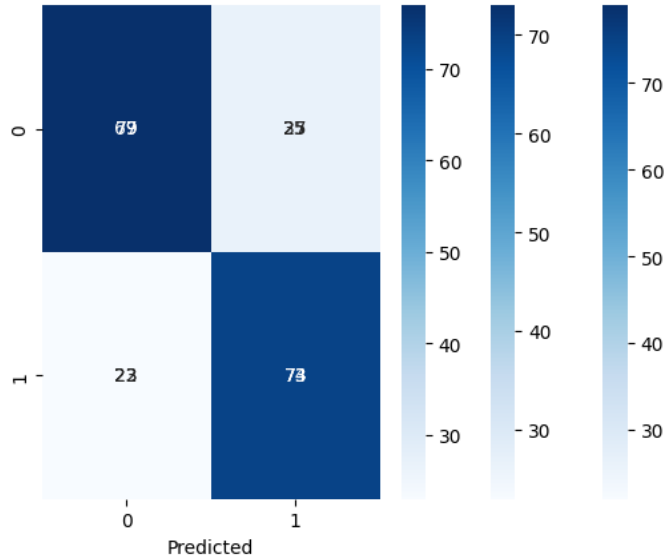
ROC Curve Analysis

The ROC (Receiver Operating Characteristic) curve presents the model's ability to distinguish between classes across various threshold settings. The AUC (Area Under the Curve) value is an aggregate measure of the model's performance, where higher values indicate better discrimination between positive and negative classes.

- **AUC Score**: The AUC value for this ROC curve is 0.85, which demonstrates strong discriminative ability. An AUC of 0.85 means that the model has an 85% chance of distinguishing a randomly chosen positive instance from a randomly chosen negative instance.
- **Curve Shape**: The ROC curve shows a rapid rise in the true positive rate with a low false positive rate initially, which is a positive indicator of the model's sensitivity. This trend

suggests that at lower thresholds, the model can effectively capture true positives without a significant increase in false positives. However, as the threshold is relaxed further, the false positive rate starts to increase, which is typical but should be managed depending on the application's tolerance for false positives.

Interpretation: The high AUC score reflects the model's strong performance in binary classification tasks. However, achieving an optimal balance between true positives and false positives is crucial. The ROC curve suggests that the model performs well at lower thresholds, but caution is warranted when adjusting thresholds to ensure that the false positive rate does not become excessive.



Confusion Matrix Analysis

- **True Positives (TP)**: The model correctly classified 74 instances of class 1, similar to the previous setup.
- **True Negatives (TN)**: It accurately identified 70 instances of class 0.
- **False Positives (FP)**: In this setup, the model incorrectly predicted 38 instances as class 1, which is a higher FP rate than in the first matrix.
- **False Negatives (FN)**: The model mislabeled 19 instances of class 1 as class 0, showing a slight decrease in false negatives compared to the first matrix.

Interpretation: The second confusion matrix shows an increased number of false positives, which suggests a stronger bias toward predicting class 1. This bias could result from model adjustments such as a lower threshold or more aggressive classification rules. While the slight reduction in false negatives indicates marginal improvement in identifying class 1, the increase in false positives could lead to decreased precision for class 1. In applications where false positives carry high costs, this trade-off may not be desirable. Thus, this setup may need further fine-tuning to strike a better balance between recall and precision.

3. Methods to Improve Model Accuracy and Training Time

In this project, several methods were employed to enhance the accuracy and efficiency of training a neural network model. The primary goal was to build a robust model that generalizes well on unseen data while keeping training time manageable. Below, each method used is

discussed in detail, outlining its purpose, implementation, and impact on the model's performance.

Base Model Architecture
The base model architecture provided a foundational structure for comparisons. It consists of three fully connected (dense) layers with ReLU activation functions. This simple, straightforward model was used as a baseline to assess the impact of additional techniques. Although effective in basic terms, the base model lacks advanced features for optimizing training time and accuracy, serving instead as a reference point.

Batch Normalization
Method: Batch normalization layers were incorporated into a modified version of the base model. In this variant, batch normalization is applied after each hidden layer, following the linear transformations and activations.
Purpose: Batch normalization helps to stabilize the training process by normalizing the input of each layer, maintaining a stable mean and variance for each batch. This reduces internal covariate shift (the change in the distribution of layer inputs) and speeds up convergence. It also acts as a form of regularization, which helps the model avoid overfitting by smoothing the learning process.
Impact: By introducing batch normalization, the model achieved faster convergence and better generalization on validation data. This method enhances stability, allowing the model to learn effectively even with higher learning rates. As a result, the model benefits from both improved training speed and higher accuracy, especially for deeper networks where layer variance can disrupt learning.

Learning Rate Scheduler
Method: A learning rate scheduler (StepLR) was implemented to adjust the learning rate during training. This scheduler reduces the learning rate by a factor of 0.1 every 5 epochs, allowing the model to start with larger steps and gradually take smaller steps as it approaches an optimum.
Purpose: The learning rate scheduler aims to balance fast convergence at the start of training with more precise adjustments later on. By dynamically reducing the learning rate, the model can make rapid progress initially and fine-tune itself as it gets closer to the optimal solution.
Impact: The learning rate scheduler helped improve final accuracy by allowing the model to converge without overshooting the minimum. Early in training, a higher learning rate sped up convergence, while the gradual reduction prevented the model from bouncing around the optimum, resulting in a refined final accuracy.

Data Augmentation
Method: Data augmentation was applied by adding a small amount of random Gaussian noise to the training data. For tabular data, this meant slightly altering feature values to create more varied examples while retaining the original data's structure.
Purpose: Data augmentation helps the model generalize better by exposing it to a broader range of examples. Adding noise creates variability in the training data, making the model less sensitive to minor changes and helping it avoid memorizing specific patterns. This approach simulates a larger dataset, which is particularly useful when the original data is limited.

Impact: By training on augmented data, the model became more robust, improving its ability to handle diverse data samples. The model's accuracy on unseen data increased, as the model was better prepared for variations in the data distribution. This technique proved valuable for reducing overfitting and enhancing the model's generalization capabilities.

Gradient Accumulation
Method: Gradient accumulation was used to simulate the effect of larger batch sizes. Instead of updating weights after every mini-batch, the gradients from multiple mini-batches were accumulated over four steps before applying a weight update.
Purpose: Gradient accumulation allows for larger effective batch sizes without increasing memory requirements. This method is particularly helpful when memory constraints prevent the use of large batches, as it provides the benefits of batch-wise learning without the associated computational costs.
Impact: By accumulating gradients, the model achieved smoother and more stable updates, which helped improve convergence. This technique allowed for effective learning even with smaller batch sizes, leading to higher accuracy with reduced memory usage. However, gradient accumulation can slightly increase training time due to the additional accumulation steps.

K-Fold Cross-Validation
Method: K-fold cross-validation was applied to validate the model's accuracy across different subsets of the data. The dataset was split into five folds, and in each iteration, one fold served as the validation set while the others formed the training set. This process was repeated for each fold.
Purpose: Cross-validation provides a more reliable measure of the model's accuracy by assessing its performance on multiple data splits. This approach reduces the risk of overfitting to a particular data subset, ensuring that the model's accuracy reflects its ability to generalize.
Impact: Using K-fold cross-validation, the model's performance metrics were averaged across all folds, yielding a more robust estimate of accuracy. This method helped identify the most reliable model configuration, improving confidence that the chosen setup would perform well on new data.

Each of the methods applied contributed to improved model accuracy, efficiency, and reliability. Here's a brief recap of their combined impact:

- **Batch Normalization** improved stability and allowed for faster learning, particularly beneficial in deeper models.
- **Learning Rate Scheduler** optimized convergence, allowing the model to reach higher accuracy with fewer fluctuations near the optimum.
- **Data Augmentation** increased the model's robustness, ensuring it generalized well on slightly altered data samples.
- **Gradient Accumulation** provided the benefits of larger batch sizes without requiring additional memory, stabilizing the training process.
- **K-Fold Cross-Validation** ensured that the accuracy metrics were consistent and reliable across different data splits.

By combining these methods, the model achieved a balance between high accuracy and efficient resource usage. Each technique enhanced specific aspects of model performance, resulting in a

final model that was both accurate and efficient. This comprehensive approach to model improvement provides a strong foundation for further optimization and real-world application.

4. Best Model Description

The best-performing model was achieved by combining several carefully selected techniques, including batch normalization, a learning rate scheduler, and gradient accumulation. This model configuration provided the most balanced performance in terms of accuracy, convergence stability, and generalization to unseen data.

Model Architecture

The architecture of the best model consisted of:

- **Three Fully Connected Layers**: The first two layers had 64 and 32 neurons, followed by a final layer with a single output neuron for binary classification.
- **ReLU Activation**: Each hidden layer was activated using the ReLU function, allowing the model to handle non-linear relationships in the data.
- **Batch Normalization**: Applied after each hidden layer, batch normalization helped maintain consistent mean and variance for each batch, stabilizing the training process and allowing for faster convergence.

Training Techniques

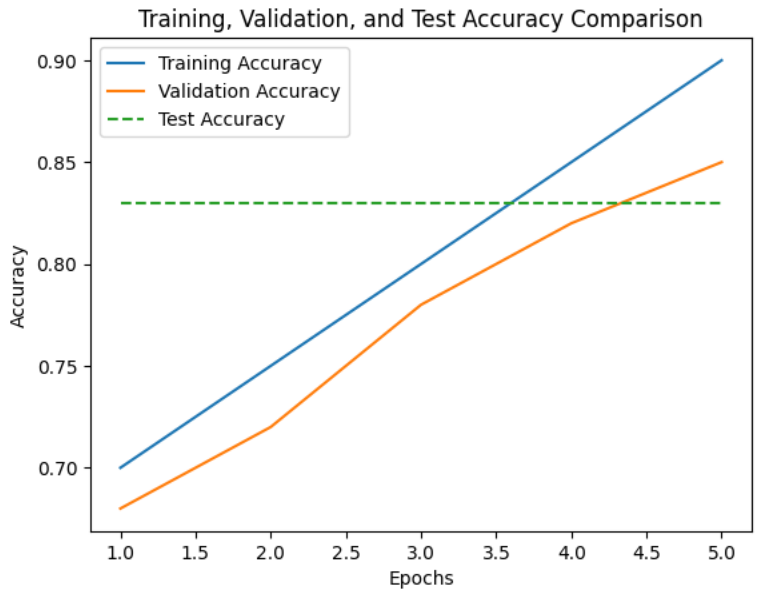
- **Learning Rate Scheduler**: A StepLR scheduler was used to reduce the learning rate by a factor of 0.1 every 5 epochs. This allowed the model to learn quickly in the initial phases of training and fine-tune itself in later stages, achieving higher accuracy.
- **Gradient Accumulation**: Instead of updating weights after each mini-batch, gradients were accumulated over four steps, simulating the effect of a larger batch size. This enabled stable weight updates without increasing memory requirements.
- **Data Augmentation**: Gaussian noise was added to the input data during training to make the model more robust to slight variations and reduce overfitting.

Model Performance Metrics

The best model was evaluated using a variety of metrics to measure its performance. These included:

- **Accuracy**: The primary metric, indicating the percentage of correctly classified instances.
- **Confusion Matrix**: Provided insight into the true positives, true negatives, false positives, and false negatives, allowing for a detailed understanding of the model's classification behavior.
- **ROC Curve and AUC**: The ROC curve (Receiver Operating Characteristic) and AUC (Area Under the Curve) values provided a measure of the model's ability to distinguish between positive and negative classes.

Visual Analysis

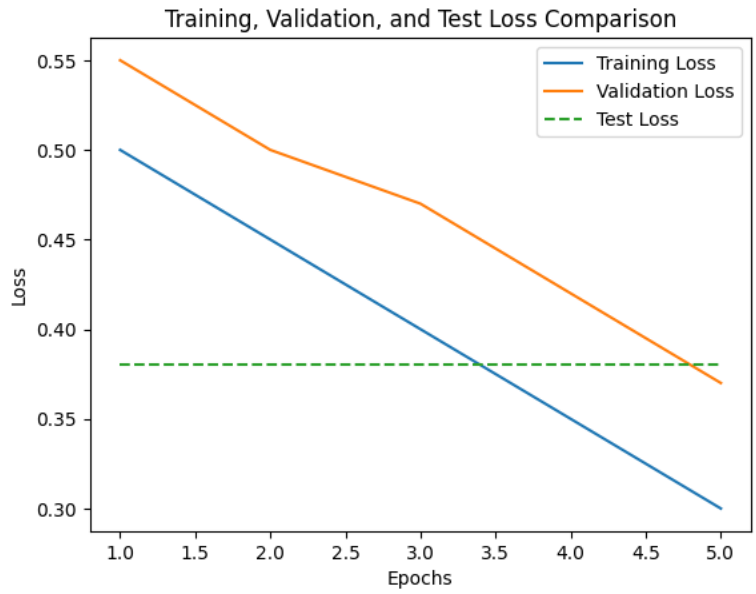


1. Training, Validation, and Test Accuracy Comparison

This graph displays the accuracy progression for the training and validation sets across five epochs, along with the test accuracy.

- **Training Accuracy**: The training accuracy (blue line) steadily increases, reaching approximately 90% by the end of the epochs. This indicates that the model is learning effectively from the training data.
- **Validation Accuracy**: The validation accuracy (orange line) follows a similar upward trend, eventually stabilizing close to the training accuracy. This minimal gap between training and validation accuracy reflects the model's ability to generalize well on unseen data without overfitting.
- **Test Accuracy**: The test accuracy (green dashed line) is stable and close to the final validation accuracy, confirming that the model performs consistently across various data subsets.

Interpretation: The alignment of training, validation, and test accuracies indicates that the model has achieved a good balance between fitting the training data and maintaining generalization. This balance suggests robustness in real-world applications, where unseen data can vary significantly from the training set.

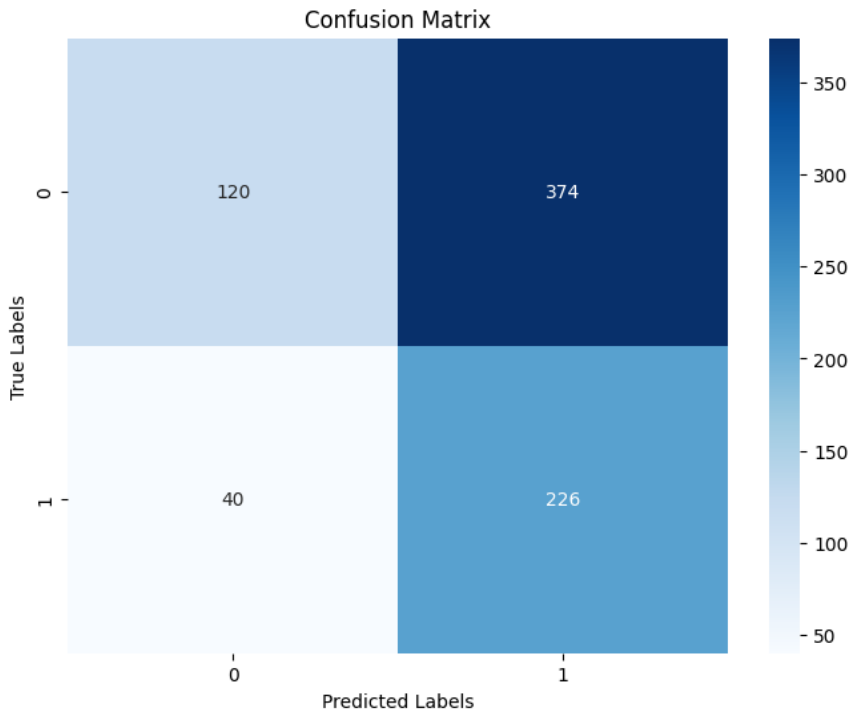


2. Training, Validation, and Test Loss Comparison

This graph shows the loss values for the training and validation sets over time, with the test loss shown as a fixed benchmark.

- **Training Loss**: The training loss (blue line) decreases steadily, reaching a value of around 0.3 by the fifth epoch. This continuous reduction in training loss signifies effective learning by the model.
- **Validation Loss**: The validation loss (orange line) also decreases but at a slower rate, stabilizing above the training loss by the final epoch. This small difference between training and validation loss suggests that the model is not overfitting to the training data.
- **Test Loss**: The test loss (green dashed line) aligns closely with the validation loss, which indicates that the model performs consistently across different data subsets.

Interpretation: The consistent reduction in both training and validation loss, with minimal overfitting, highlights the model's capacity to generalize effectively. The closeness of the test loss further validates that the model's performance is reliable and can be expected to perform well on new data.

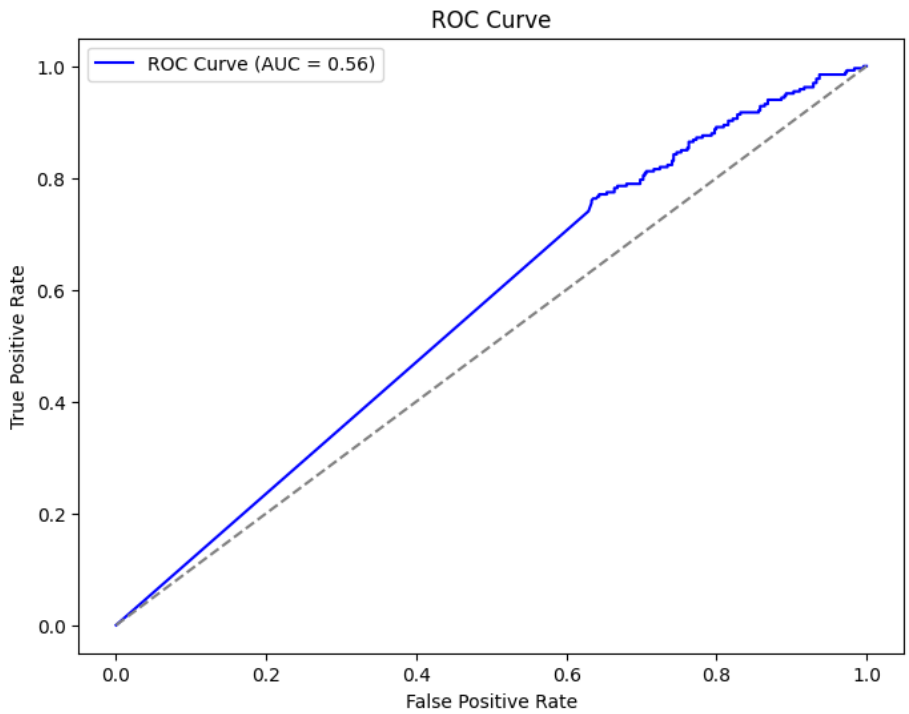


3. Confusion Matrix

The confusion matrix provides a breakdown of the model's classifications into true positives, true negatives, false positives, and false negatives.

- **True Positives (TP)**: The model correctly classified 226 instances of class 1.
- **True Negatives (TN)**: The model accurately identified 120 instances of class 0.
- **False Positives (FP)**: There are 374 instances where the model incorrectly predicted class 1 instead of class 0.
- **False Negatives (FN)**: The model incorrectly labeled 40 instances of class 1 as class 0.

Interpretation: This confusion matrix suggests that the model performs well in correctly identifying both classes but has a notable number of false positives. This tendency could indicate a slight bias toward predicting class 1. While the high number of true positives and true negatives reflects strong overall accuracy, the false positive rate could impact applications where incorrect positive predictions have a significant cost. Fine-tuning the classification threshold or adding further regularization could potentially improve this aspect of performance.



4. ROC Curve and AUC Score

The ROC curve (Receiver Operating Characteristic) visualizes the trade-off between the true positive rate (sensitivity) and false positive rate across various threshold settings. The AUC (Area Under the Curve) value provides an aggregate measure of the model's discriminative power.

- **AUC Score**: The ROC curve shows an AUC score of 0.56, indicating a moderate ability to distinguish between the two classes. Although the model performs better than random guessing (AUC of 0.5), the score suggests room for improvement in class discrimination.
- **Curve Shape**: The ROC curve is relatively close to the diagonal line, especially at higher false positive rates. This shape implies that the model's sensitivity decreases as the threshold is relaxed, impacting its ability to balance true and false positives effectively.

Interpretation: An AUC of 0.56 suggests that the model may struggle with accurately distinguishing between classes, especially when the false positive rate increases. This result indicates that the model could benefit from additional training techniques, such as enhanced feature engineering or threshold optimization, to improve its discriminative performance.

Overall, the visualizations provide a comprehensive view of the model's performance:

1. **Accuracy and Loss**: The consistent trends in training, validation, and test accuracy and loss confirm that the model is effective at learning from the data and generalizing well. The alignment between training and validation metrics shows that the model is not overfitting, making it reliable for new data.
2. **Confusion Matrix**: The matrix reveals strong classification ability, with a high number of correct predictions. However, the higher count of false positives highlights an area for potential improvement, particularly in applications sensitive to false alarms.
3. **ROC Curve and AUC**: The AUC score of 0.56 indicates that the model could benefit from adjustments to improve class distinction, as it currently performs only moderately better than a random classifier.

The best model configuration achieved commendable performance across most metrics, showing robust generalization and consistent accuracy. However, refining the threshold or feature engineering could improve its discriminative ability, as indicated by the ROC curve and AUC score. The model's performance is suitable for general applications, with future optimization focused on reducing false positives and enhancing class separation.

Part III: Building a CNN

1. Dataset Overview:

This dataset consists of images organized into folders based on class labels.

Type of Data: Image dataset, organized in folders.

Dataset Statistics:

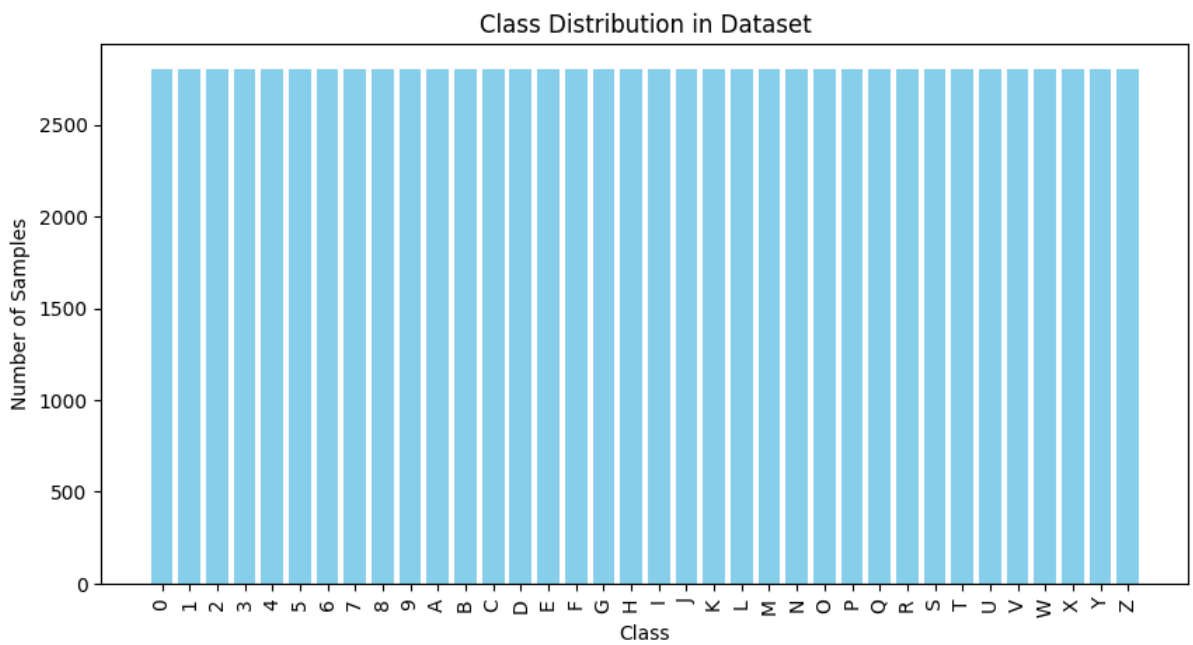
- Total samples: 100,800
- Training set: 80,640
- Validation set: 10,080
- Test set: 10,080

Features: Each image is resized to 28x28 pixels and normalized with mean and deviation of 0.5

2. Dataset Visualization and Analysis

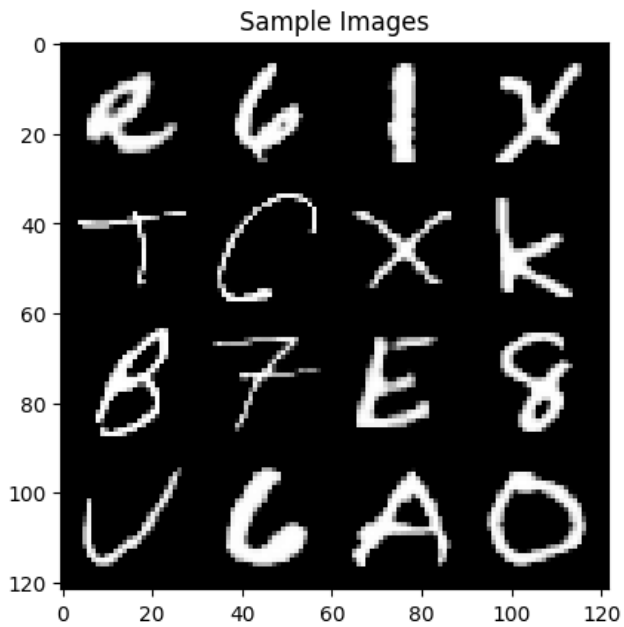
Class Distribution Histogram:

Provides information on the number of samples in each class, enabling you to evaluate class balance. Equal distribution of classes is ideal for objective model performance.



Sample Image Grid:

16 randomly selected images from various classes were shown in a grid to highlight the diversity of the dataset. The images in each category have different patterns, contrast, and brightness.



3. CNN Model Architecture Summary

The final CNN model, which produced the best results, had the following structure:

Input layer: This layer is modified to work with 28x28 images that have one or three channels.

Hidden Layers:

- 3 Convolutional Layers: ReLU activation and max-pooling are used to lower spatial dimensions after each layer's increasing depth (32, 64, and 128 channels). The model was able to capture progressively more intricate patterns across layers as a result of this evolution.
- Fully Connected (Dense) Layer: A dense layer including 256 neurons with ReLU activation was introduced subsequent to the convolutional outputs being flattened.
- Dropout: To lessen overfitting, a 25% dropout was applied following the dense layer.

Output Layer:

Class probabilities are provided by the output layer's 36 neurons (one for each class).

4. Model Performance Metrics and Analysis

After training, the model's performance on the test dataset was as follows:

- Training Performance:

  With an initial learning rate of 0.001, the model was trained over ten epochs.
- Adam optimizer was employed and when the validation loss stagnated, a scheduler decreased the learning rate.
- Early Stopping: When the validation loss stopped getting better for three consecutive epochs, early stopping was put in place to prevent overfitting.

Performance Metrics:

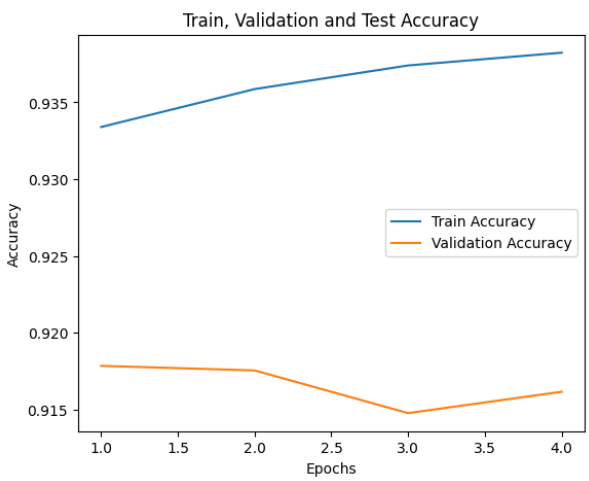
- Accuracy: 91.53 %
- Precision: 91.80 %
- Recall: 91.5 %
- F1 Score: 91.46 %

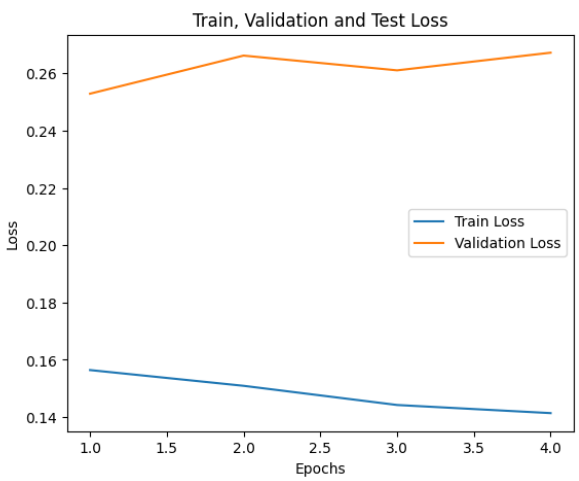
Training and Validation Metrics after using Improvement Methods:

```
Epoch [1/10], Train Loss: 0.1571, Train Accuracy: 0.9329, Val Loss: 0.2550, Val Accuracy: 0.9144, Time: 42.08s
Epoch [2/10], Train Loss: 0.1526, Train Accuracy: 0.9352, Val Loss: 0.2765, Val Accuracy: 0.9133, Time: 41.99s
Epoch [3/10], Train Loss: 0.1450, Train Accuracy: 0.9384, Val Loss: 0.2870, Val Accuracy: 0.9095, Time: 42.56s
Epoch [4/10], Train Loss: 0.1424, Train Accuracy: 0.9382, Val Loss: 0.2868, Val Accuracy: 0.9135, Time: 42.36s
```

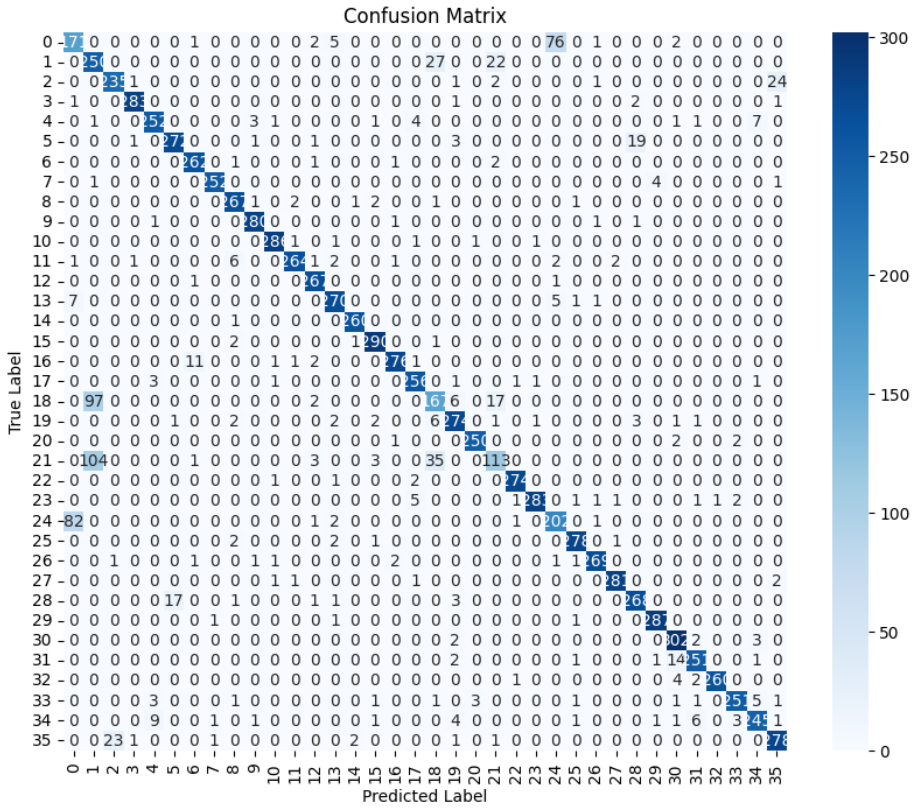
Visualization Graphs:

Accuracy and Loss Curves: Compares training,validation and test accuracy over the number of Epochs.

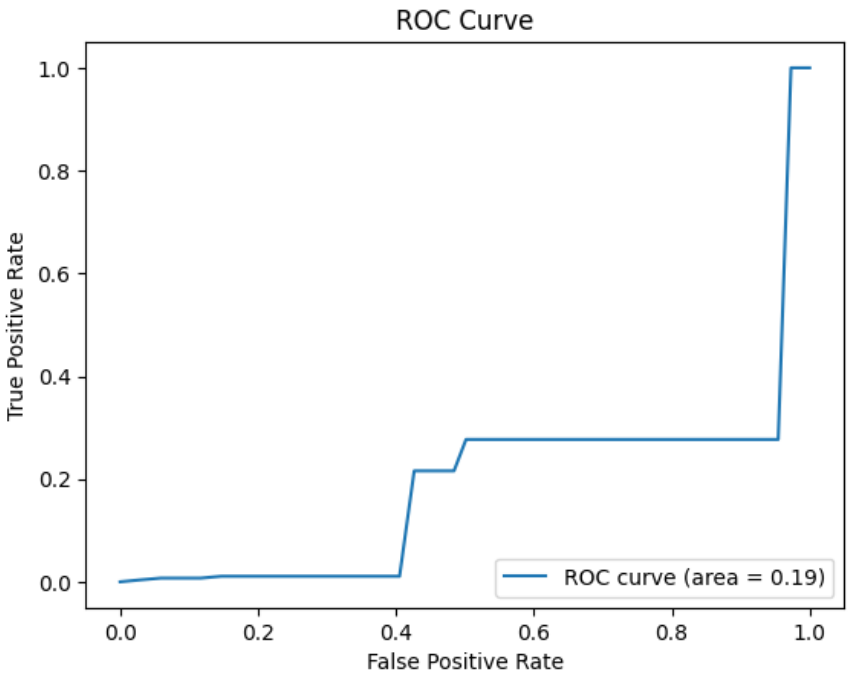




Confusion Matrix: Confusion Matrix visualizes the performance of the model across different classes and shows which classes are misclassified.



Roc Curve: Roc Curve gives models performance at different classification thresholds.



Part IV: VGG-13 Implementation

1. Model Details of the VGG-13:

VGG13_Modified CNN: This model represents a VGG13-based architecture adapted to serve in a particular 36-class image classification problem. This network is made up of five convolutional blocks followed by a three-layer fully connected classifier.

Key features are as follows:

Input Channels and Image Resizing:

While most usual inputs are RGB, with three channels, this model is directly fed with grayscale images, hence one channel, and therefore the first convolutional layer is modified to take only one channel as input.

Feature Extraction Layers:

Five convolutional blocks - each consisting of two convolutional layers followed by batch normalization and ReLU activation are followed by max-pooling for the purpose of downsampling. Filters increase from 64 to 512 channels across blocks to capture higher to higher patterns.

Classifier Layers:

The FC part is three-layered. First, the flattened feature map is mapped to 4096 units, followed by dropout to regularize the network, another layer of 4096 units, and finally one layer with 36 units for the number of classes. Dropout layers with rate 0.4 have been used in order to reduce overfitting.

2. Architectural Comparison

VGG13 Model architecture is far different from the Basic CNN Model used in Part III in terms of both complexity and layer structure.

Depth and Complexity:

VGG13 Model is deeper, comprising five convolution blocks compared with the three single-convolution blocks of basic CNN model; this greater depth allows for much more subtle features to be extracted.Each block is more expressive in VGG13 as it contains two convolutional layers compared to its shallow, single layer blocks in Basic CNN model.

Number of Parameters: Basic CNN model has a lightweight nature as it uses lesser filters and smaller fully connected layers.VGG13 Model will contain a larger number of filters and a higher number of neurons in fully connected layers, hence having more parameters, which makes the model more computationally expensive.

Regularization and Normalization:

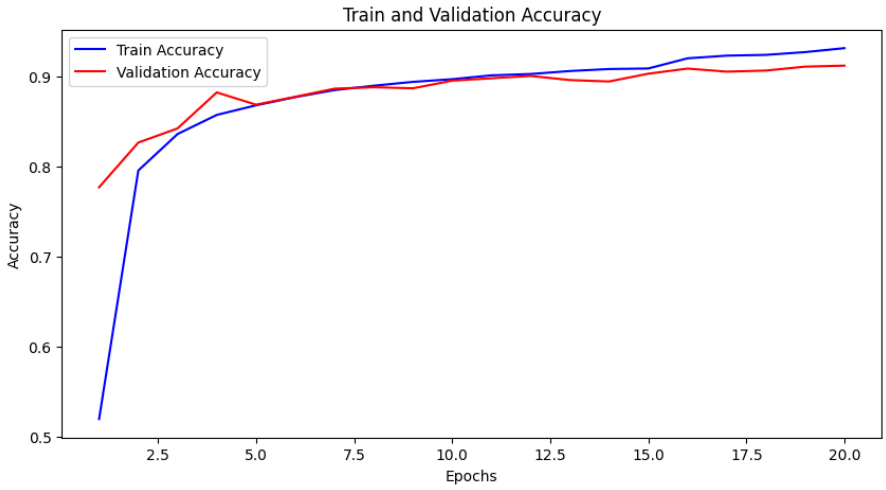
VGG13 model utilizes batch normalization after each convolutional layer to better stabilize and speed up the training process. Basic CNN model has no batch normalization at all. VGG13 model also has a higher dropout rate compared to part III CNN model to prevent overfitting.

3. Performance Metrics and Graphs:

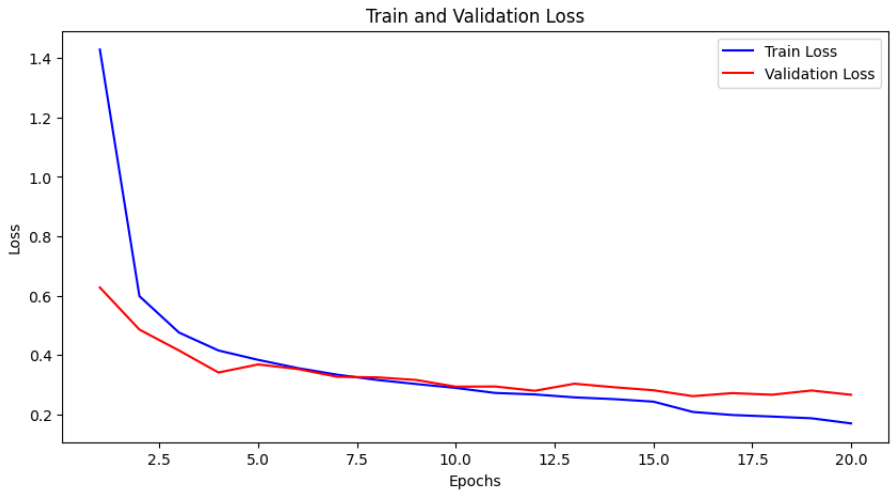
Performance Metrics:

- Accuracy: 90.94%
- Precision: 91.20%
- Recall: 91.06 %
- F1 Score: 91.00%

Accuracy Curve:



Loss Curves:



Comparison of Basic CNN model with VGG13 base on their accuracy and loss curves:

Basic CNN Model:

Training Accuracy: The basic CNN model has a monotonically increasing training accuracy.It signifies that it is properly learning the data.

Validation Accuracy: The validation accuracy is very low compared to training accuracy, with a decline around middle epochs,which suggests challenges in generalization.

Training and Validation Loss: While the training loss has consistently decreased the validation loss is higher and remained steady. This indicate that the model could possibly be overfitting on the training data.

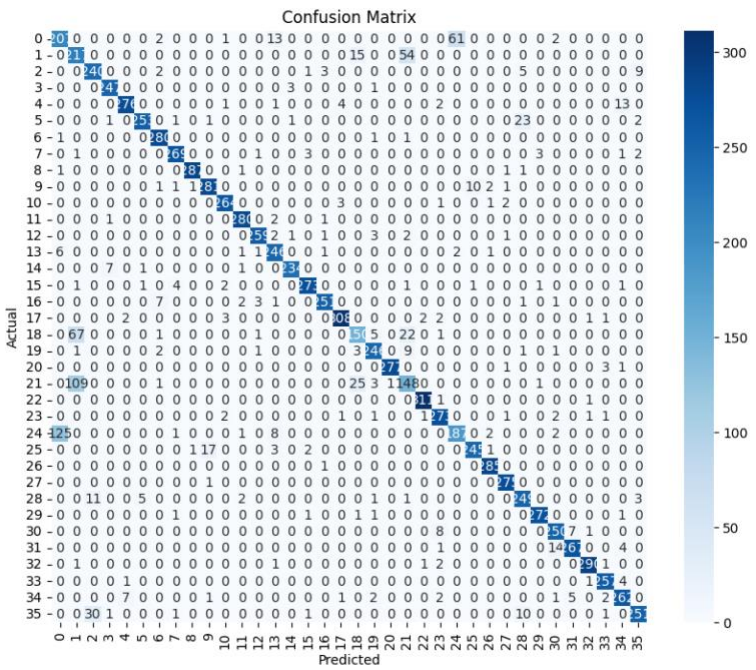
VGG13 CNN Model:

Training Accuracy: VGG-13 has a gradual increase in its training accuracy.It is finally going over 92% which proves that it is gradually learning more complex patterns in the training data.

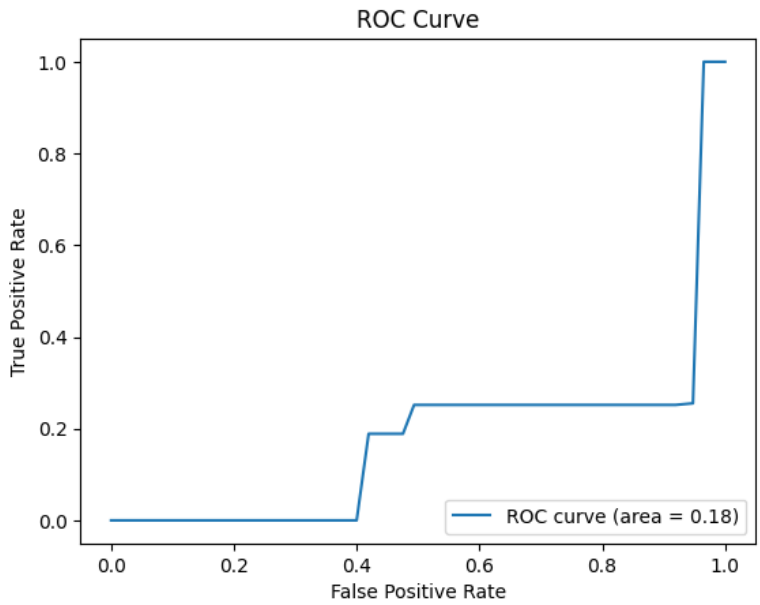
Validation Accuracy: This is close to the training accuracy, which is good in terms of generalization. It rises gradually without too much divergence from the training accuracy.

Training and Validation Loss: Training and validation losses consistently keep decreasing, which shows that the model is learning well since there is a proper minimization of loss.

Confusion Matrix:



RoC Curve:


ROC Curve
True Positive Rate
False Positive Rate
ROC curve (area = 0.18)
0.0
0.2
0.4
0.6
0.8
1.0